AI PRODUCT MANAGEMENT

14 Principles of Building AI Agents (Learned tl Hard Way)

What I learned by building 50+ AI agents and copying the Multi-Agent Research Syst Anthropic. Best practices and mistakes to avoid.

PAWEŁ HURYN

JUL 12, 2025 • PAID

68 5 Share

Hey, welcome to the **archived edition** of The Product Compass newsletter.

Every week, I share actionable tips, resources, and insights for PMs.

Here's what you might have missed recently:

1. The Ultimate AI PM Learning Roadmap
2. The Ultimate Guide to AI Agents for PMs
3. Introduction to AI Product Management
4. Base44: A Brutally Simple Alternative to Lovable
5. How to Figma → Jira epics and stories in 10 min.

Consider subscribing or upgrading your account for the full experience:

Updated: 11/2/2025 (added experiment results for GPT-5)

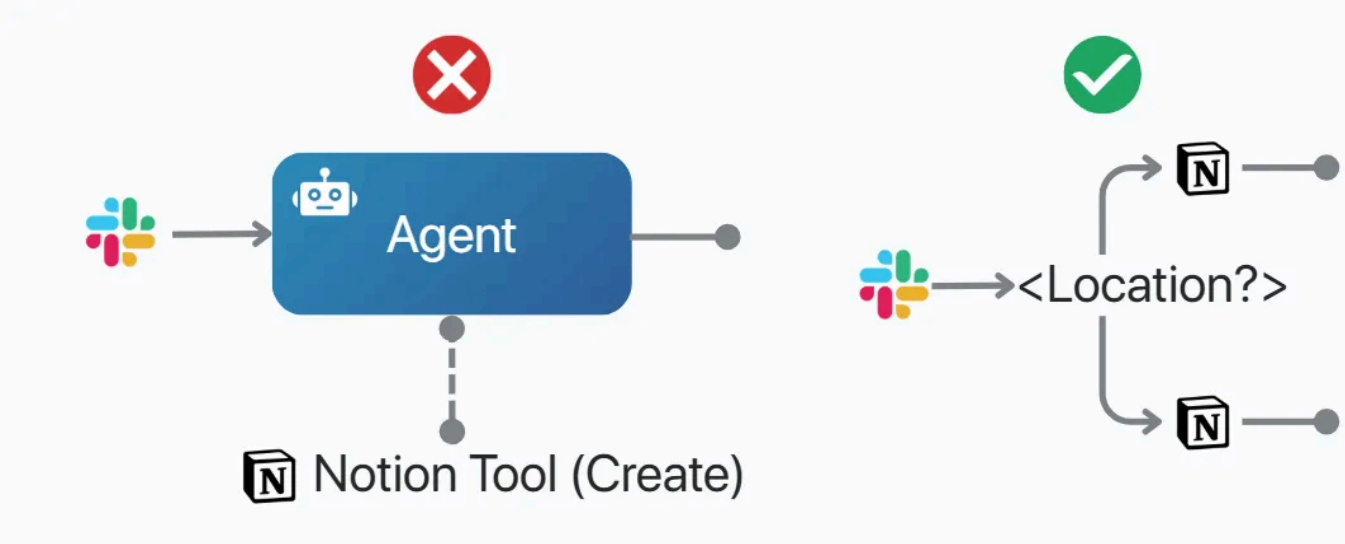
In the recent months, I've built 50+ AI agents, experimented with 7+ agentic frameworks, and copied the Multi-Agent Research System by Anthropic.

Here's what I learned (best practices and mistakes to avoid):

1. Don’t Use Agents If You Don't Have To

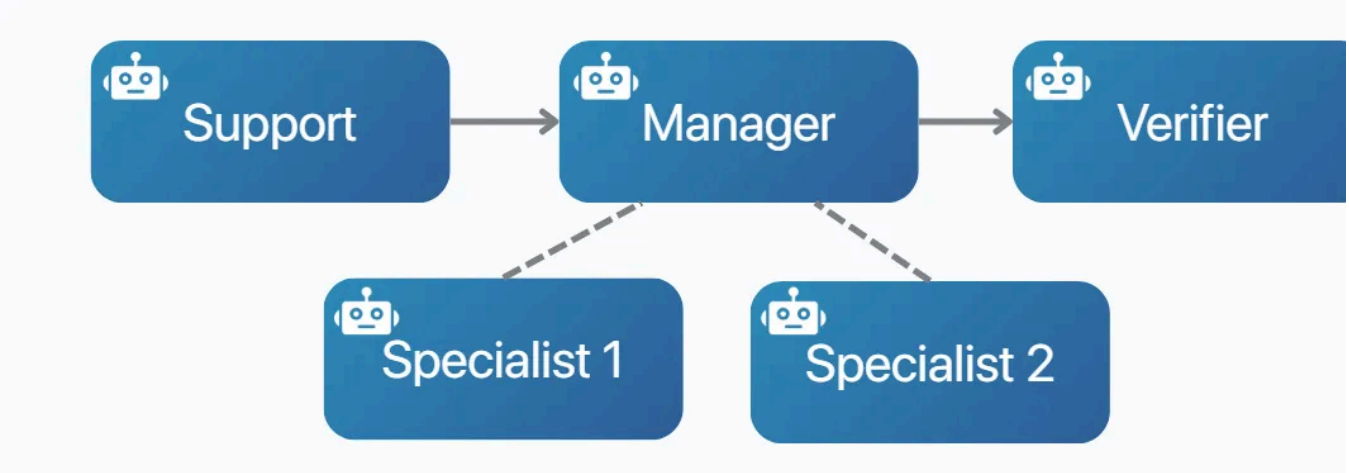
Nobody cares if it's an AI agent or a simple script, as long as it works. A good old if/else is faster, cheaper, and more reliable. And it's often all you need.

Save the agents for when you really need them. They might easily become a liab



2. Small, Specialized, and Decoupled

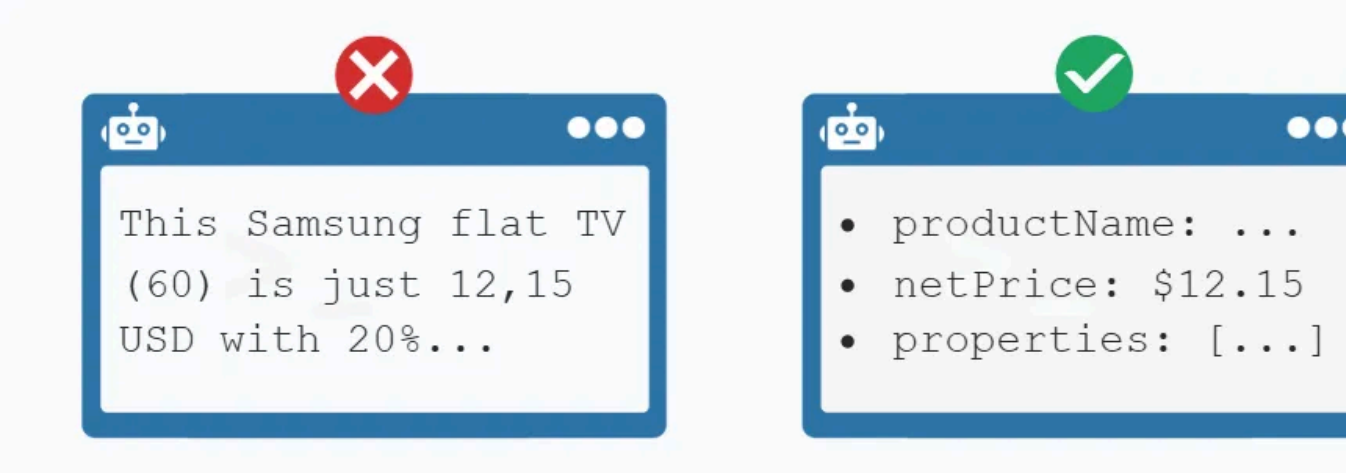
Think "team of specialists," not "one agent to rule them all." A planner plans. A summarizer summarizes. A verifier checks. Decoupled agents are cheaper to rur easier to test and fix, and way more predictable.



3. Enforce Structured Output

I've learned that text is a mess to deal with. JSON is easier to debug, cheaper to and acts like a contract between agents.

Bonus: you can validate it automatically and stop errors before they spread.



4. Explain the Why, Not Just the What

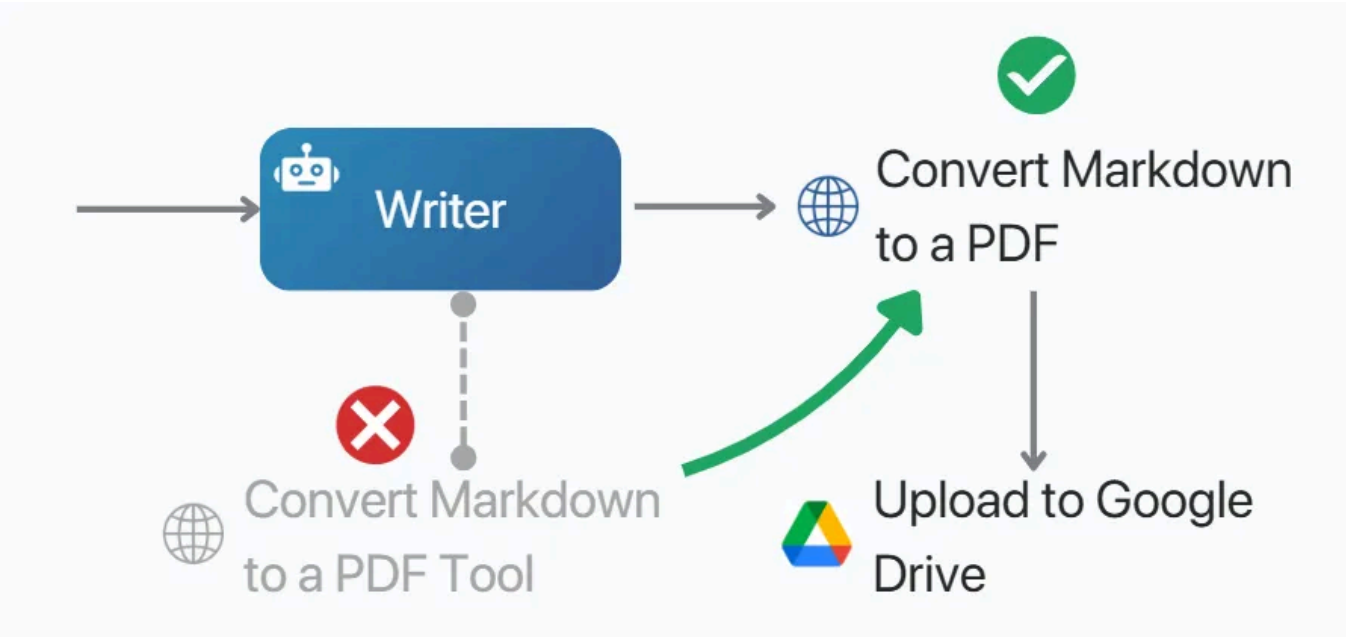
I've discovered that anthropomorphizing AI works in many contexts. Here, lead v context not control.

When delegating a task, don't just define the objective. Explain why it matters ar provide the context in which you need it.

This helps AI agents make better decisions with shorter prompts.

5. Orchestration > Autonomy

Autonomy sounds great, but what you need more in real life is predictability. Mov known logic (if/then, loops, retries, known procedures) out of agent prompts anc the orchestration layer.

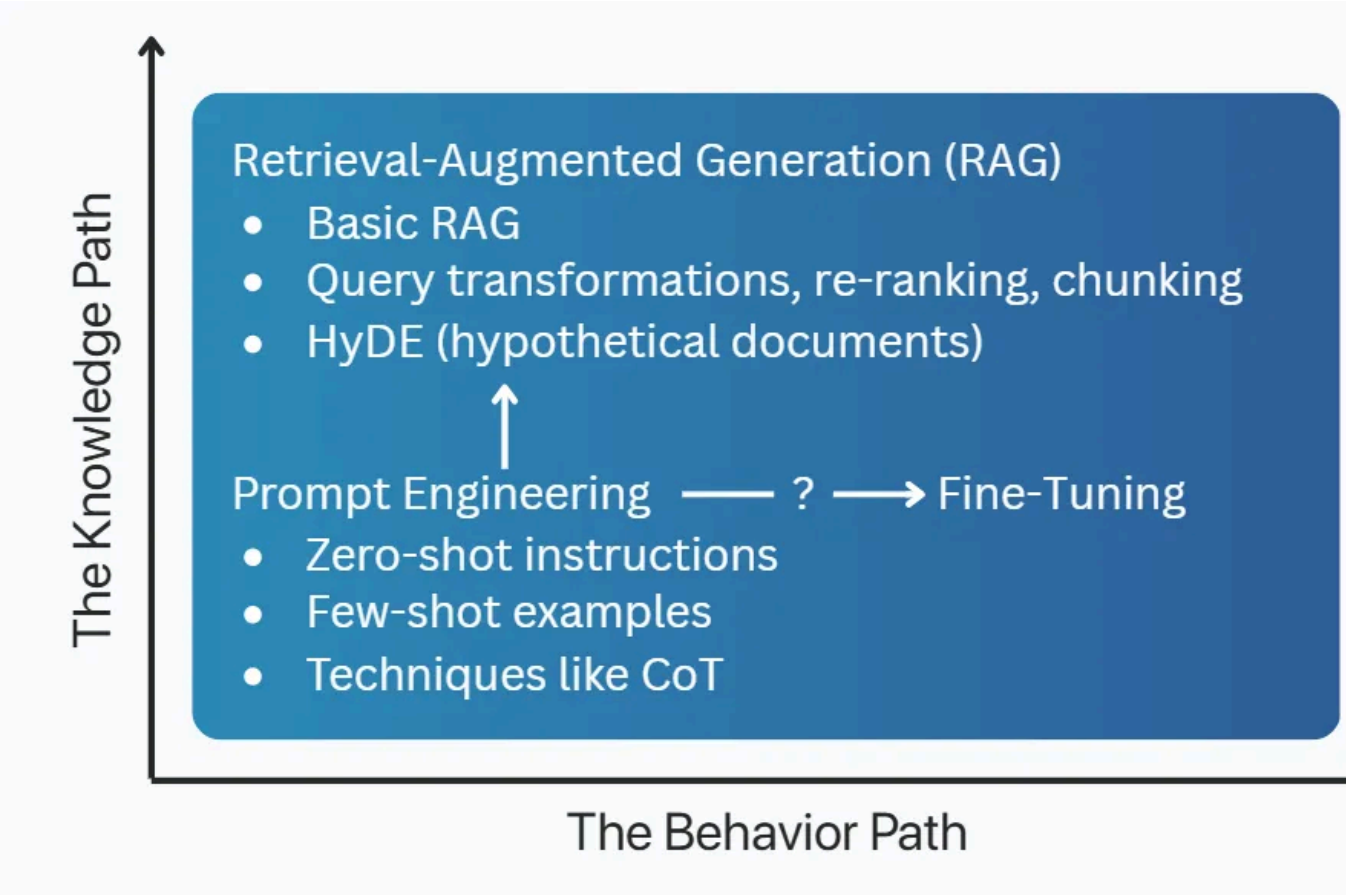


6. Prompt Engineering > Fine Tuning

Before you jump to fine-tuning, ask: Why is the model failing?

- If it's missing facts → try RAG.
- If it's wrong formatting or doesn't follow your brand style → maybe fine-tune

80% of the time, it's just a prompt problem.



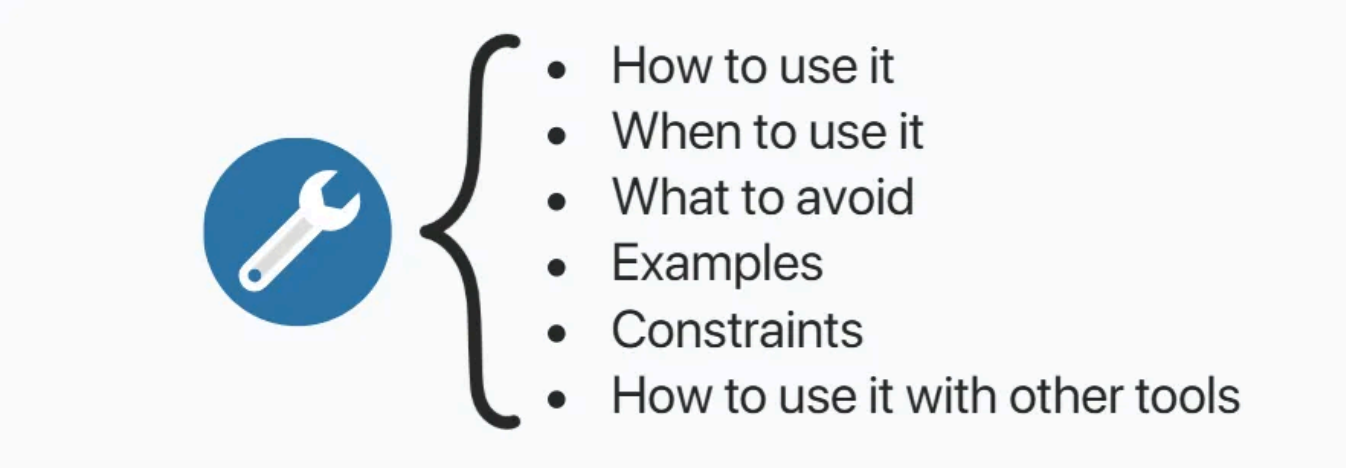
Source: AI Product Management Certification by Miqdad Jaffer (OpenAI)

7. Double Down on Tool Descriptions

Treat tool descriptions as micro-prompts that guide agents' reasoning. Unfortun descriptions provided by MCP servers are often insufficient and do not consider specific domain context.

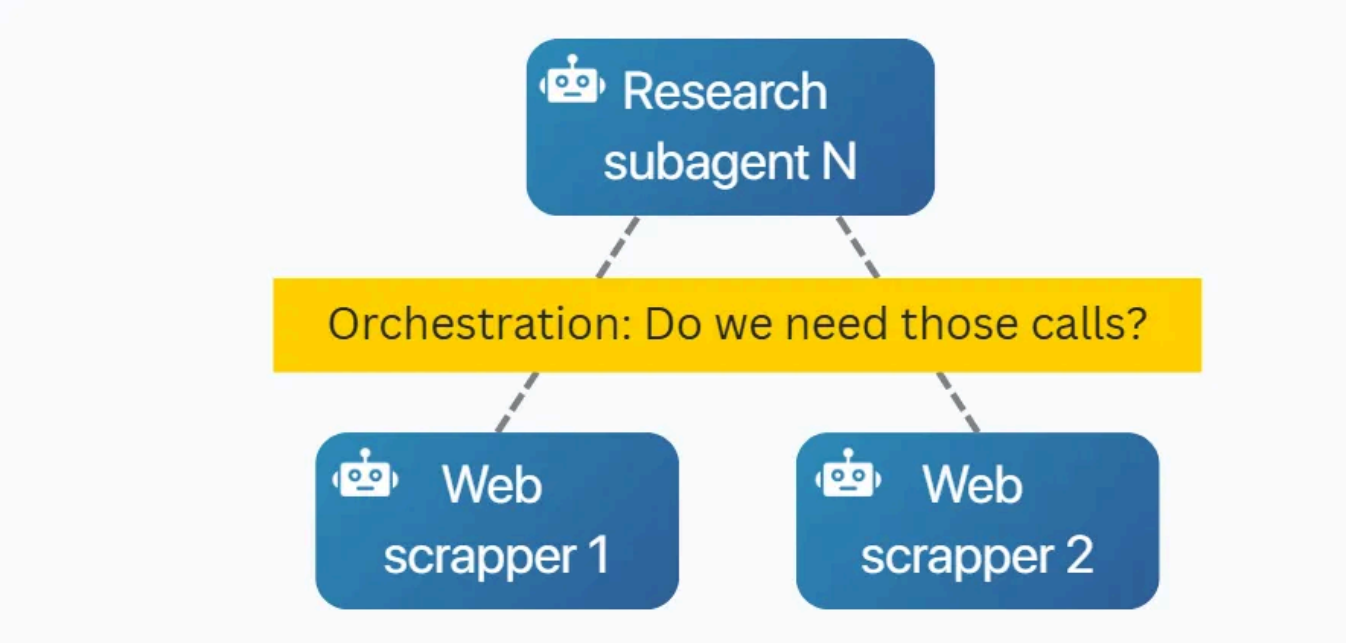
Tell the agent when and why to use the tool, what to avoid, and include example

Practical tip: Explain how tools can work in combination (e.g., for the Trello MCP agent had to list boards, get available lists inside a selected board, etc.) I usually those instructions in agent prompts.



8. Cache Like You Mean It

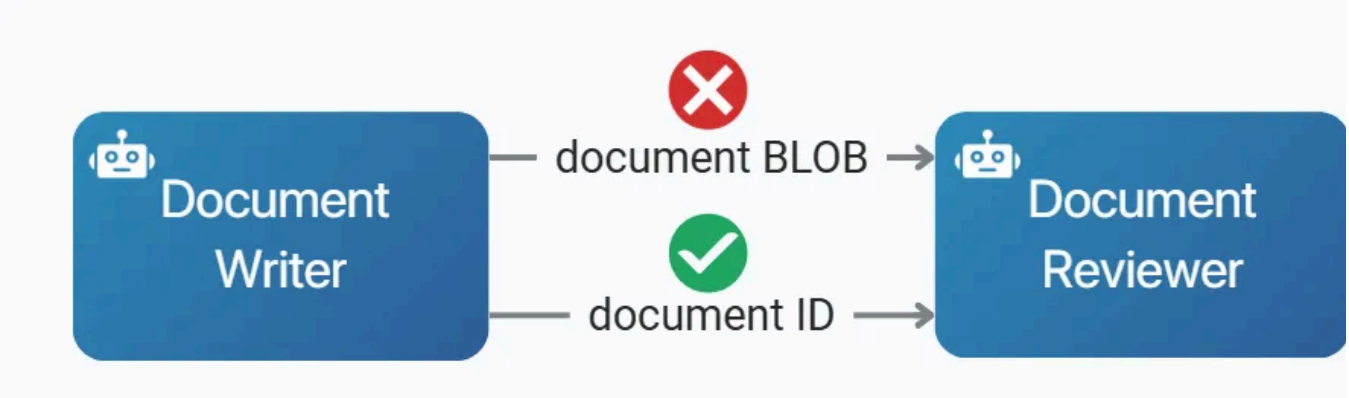
Often, an agent runs the same task on the same data over and over, like when scrapping a website. Cache responses (e.g., hash of agent ID + input) to reduce latency and API costs.



9. Use Shared Artefacts

Do you send documents you collaborate on as attachments? Of course, not. Sim empower your agents to collaborate by co-editing shared docs, plans, or code.

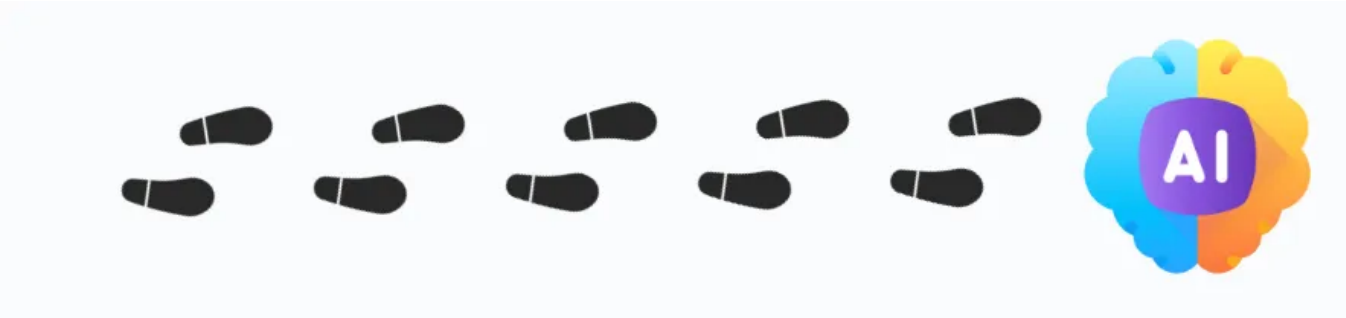
Also, often, the next agent doesn't even need the content of the artefact.



10. Log Everything (Seriously)

No logs = no learning. Track everything: inputs, outputs, retries, tool calls, agent thoughts. Add your own app-specific dimensions (e.g., customer type, use case

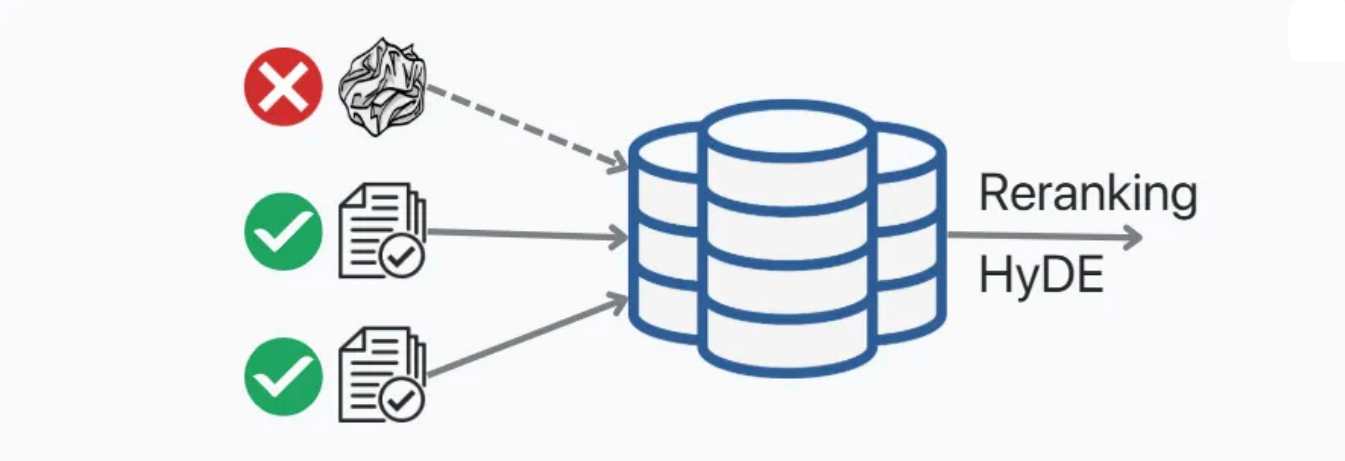
Then analyze errors and design evaluators.



11. RAG Retrieval Might Not Be the Problem

People keep discussing advanced retrieval strategies like chunking, reranking, o HyDE (hypothetical documents). But when building chatbots, I learned that the r problem with RAG might be **data curation**.

A thankless, manual job nobody wants to talk about.

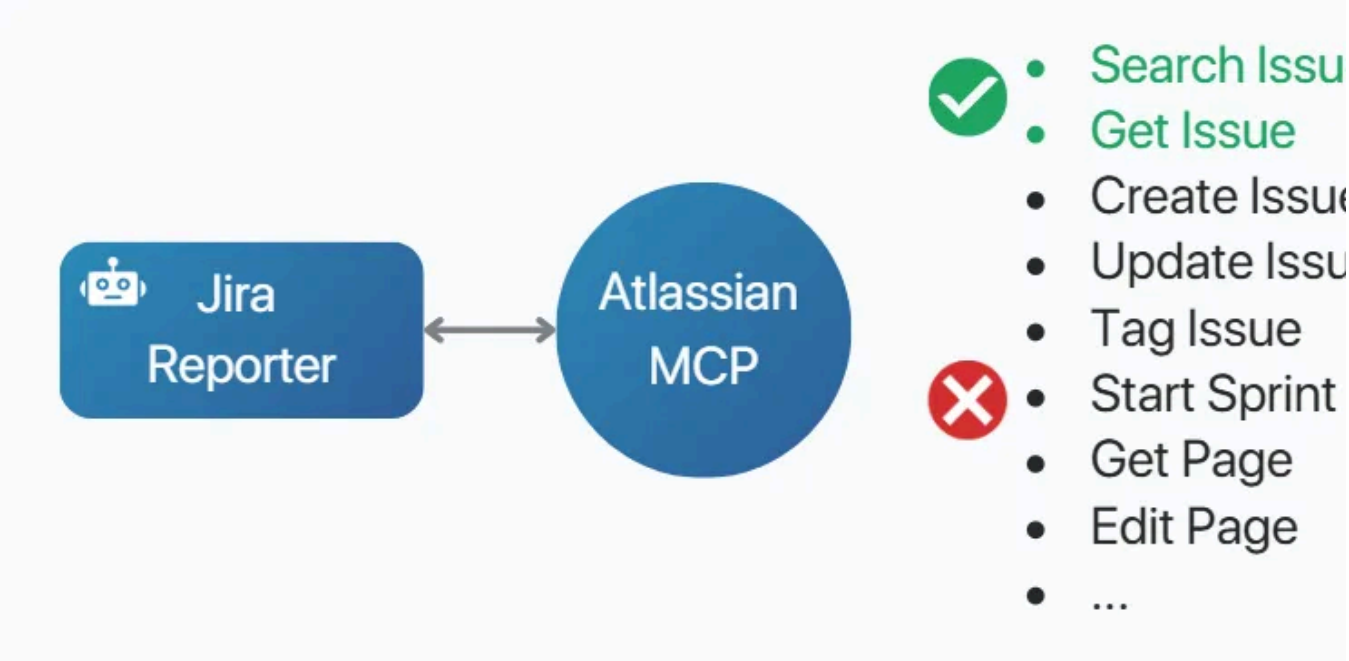


Practical insight: For production, 90% of the time you need a Hybrid RAG, in wh you combine semantic search (vector search on chunks) with filtering by metada (data, author, and document type).

12. Limit the Tools an Agent Can Access

Connecting an agent to three MCP servers, each with 30+ tools might sound gre Until you analyze the traces. Your agent is bombarded with a ridiculously large c that contains all the tools with descriptions and examples. A more obvious reaso restrict tool access is security.

Practical insight: It's possible with built-in n8n MCP Client Tool node (SSE), but supported by the MCP Community node yet.

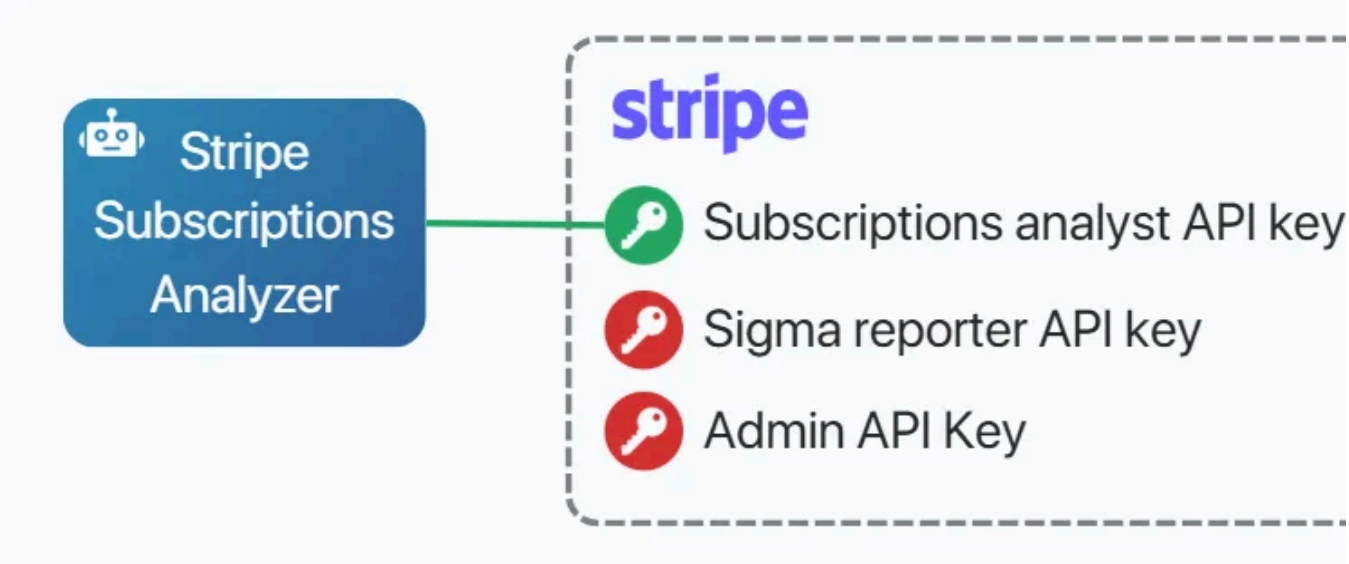


13. Create Scoped API Keys

When building n8n workflows with Stripe integration I realized every agent would full access to all Stripe operations.

The pattern I came up with:

1. Define roles for each external system. For Stripe, this might be:
    a. Subscriptions analyst (read-only access to customers and invoices).
    b. Sigma reporter (can run SQL Sigma reports).
2. Create a dedicated API key for each role.
3. Assign dedicated API keys to agents.



14. Create a Prompt Repository

When I started building my B2B SaaS PoC, I realized I can't let prompts be rando strings in my workflows.

A way better approach, natural for engineers, is storing prompts in a version-controlled repository (e.g., Git). Then, you can access them, for example, by nam version.

To simplify, starting with a simple Google Sheet you can read from n8n might be enough.

Key	Version	Prompt
PRD_Generate	3.0	## Role \nYou are an expert (...)
PRD_Generate	2.0	You are an expert PM. Generate (...)
PRD_Generate	1.0	Generate a PRD from {user_input}
Market_Research	2.0	## Role \nYou are a veteran(...)
Market_Research	1.0	Perform market research. Request: (..

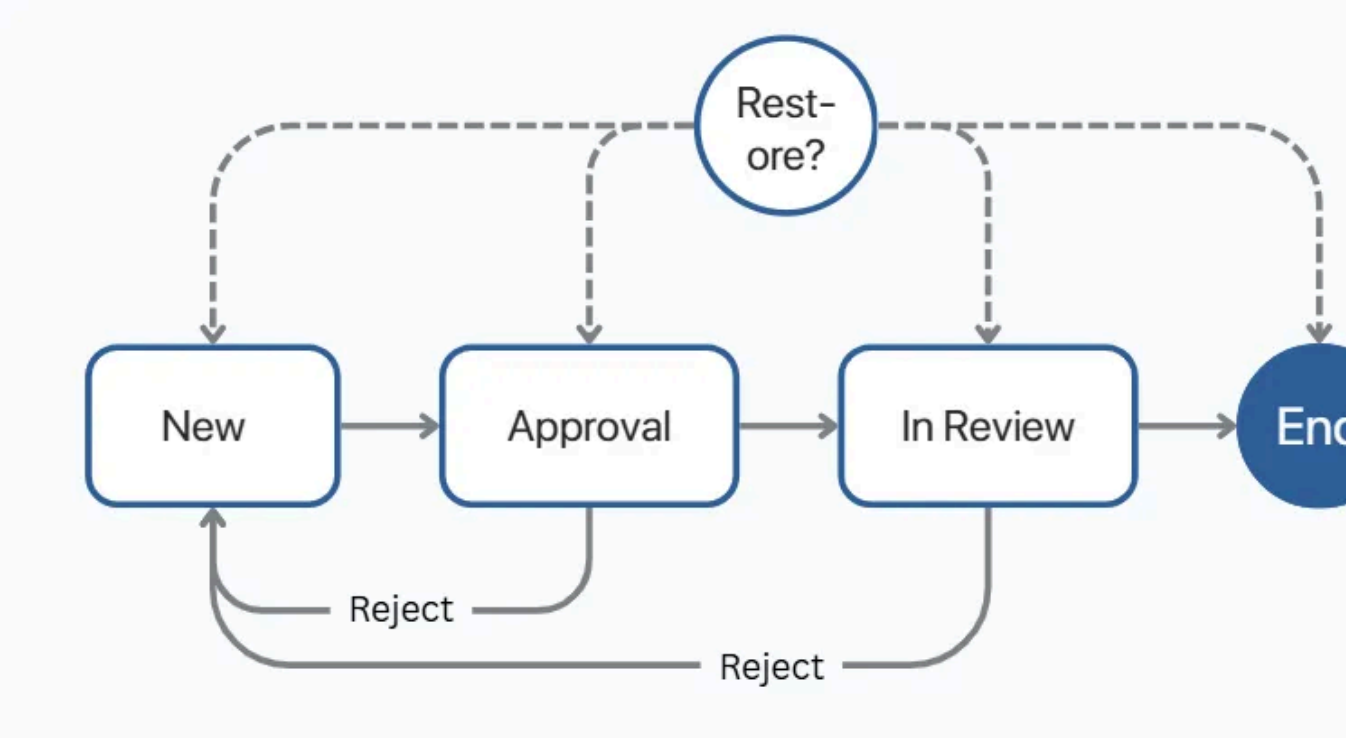
15. Bonus: Build State Machines For Large Agen Workflows

I learned this rule long before AI, when automating business processes and partn with providers like Nintex Workflow. I haven't applied it yet to agentic workflows, it's too important to skip.

Most long-term, complex workflows should be designed as state machines with:

- Clearly defined states (e.g., for an invoice: submitted, description, awaiting approval, financial control, under approval, approved, paid, rejected)
- Transitions between them (e.g., approval → financial control).

Practical insight: Design your workflows so you can run them again while restor the previous state. This helps a lot when things go sideways.



16. Bonus: My Tests And 3 More Best Practices

I spent 12 hours testing 9 LLMs for building AI agents:

- You might easily save up to 83% on costs.
- Reasoning models are not the best.
- Autonomy break fast. A real moat is orchestration.

Here's everything you need to know:

The task assigned to agents:

1. Create a new list inside Kanban (1 Trello board available)
2. Search the web to find the recent news about Amazon
3. Add all the search results to the Kanban list

Completing this task required:

- Preparing a simple plan
- Using multiple tools in the right order
- Adjusting the plan if anything goes wrong
- Operating autonomously (the system prompt gave no clues about the proce

Aggregated results (multiple runs):

Which LLM Should Power Your AI Agents?

System prompt (high autonomy)

You are a helpful assistant. When the user gives you a task:

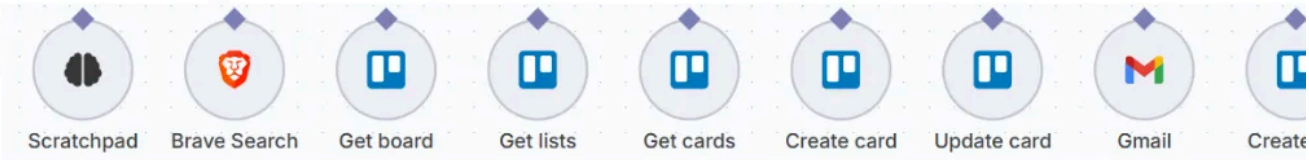
1. Make a plan. What information and tools and in which sequence do you need?
2. Then start executing that plan step-by-step
3. After each step, review where you are, decide whether to update the plan, and continue

User request

Think step by step:

1. Create a new list inside {url} Kanban board with name equal to your specific LLM model
2. Next, use web search to find the recent news ab Amazon
3. Finally, add ALL the search results to the Kanbar you created earlier

Available tools



Model comparison (10x tests average)

Model	GPT 5	GPT 4.1	4.1-mini	Grok 4	Gemini 2.5 Pro	Claude Sonnet 4	GPT o3-mini	GPT 5 mini
Opened the Kanban board	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Ye
Created a new list	Yes	⚠ 2nd attempt	⚠ 2nd attempt	⚠ 2nd attempt	⚠ 2nd, duplica-ted lists	⚠ 2nd attempt	⚠ 2nd attempt	⚠ 2nd dupli ted li
Used Brave Search	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Ye
Created all cards	Yes (10 news)	Yes (10 news)	Yes (10 news)	Yes (10 news + 6 videos)	Yes (10 news)	⚠ Only 3 cards	⚠ Only 6 cards	⚠ On 6 ca
Created cards in pararell	Yes	Yes	Yes	Yes	Yes	⚠ No, 1 at a time	⚠ No, 1 at a time	⚠ No, at a t
Used the right Kanban list	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Ye
Time	4m 13s	40s	33s	3m 17s	59s	45s	41s	2m 1
Tokens	66K	39K	41K	149K	68K	85K	74K	79

Best practices

- Use GPT 5 ~~non-reasoning LLMs~~ by default
- Provide a scratchpad for AI agent to make notes
- Or use solution like Sequential Thinking MCP to control the process

Insights

- GPT 5 >> 4.1 > 4.1-mini > Other models
- GPT-5 is way more error-resilient
- 4.1-mini is 83% cheaper than 4.1
- Above 3-5 tools, every model makes mistakes

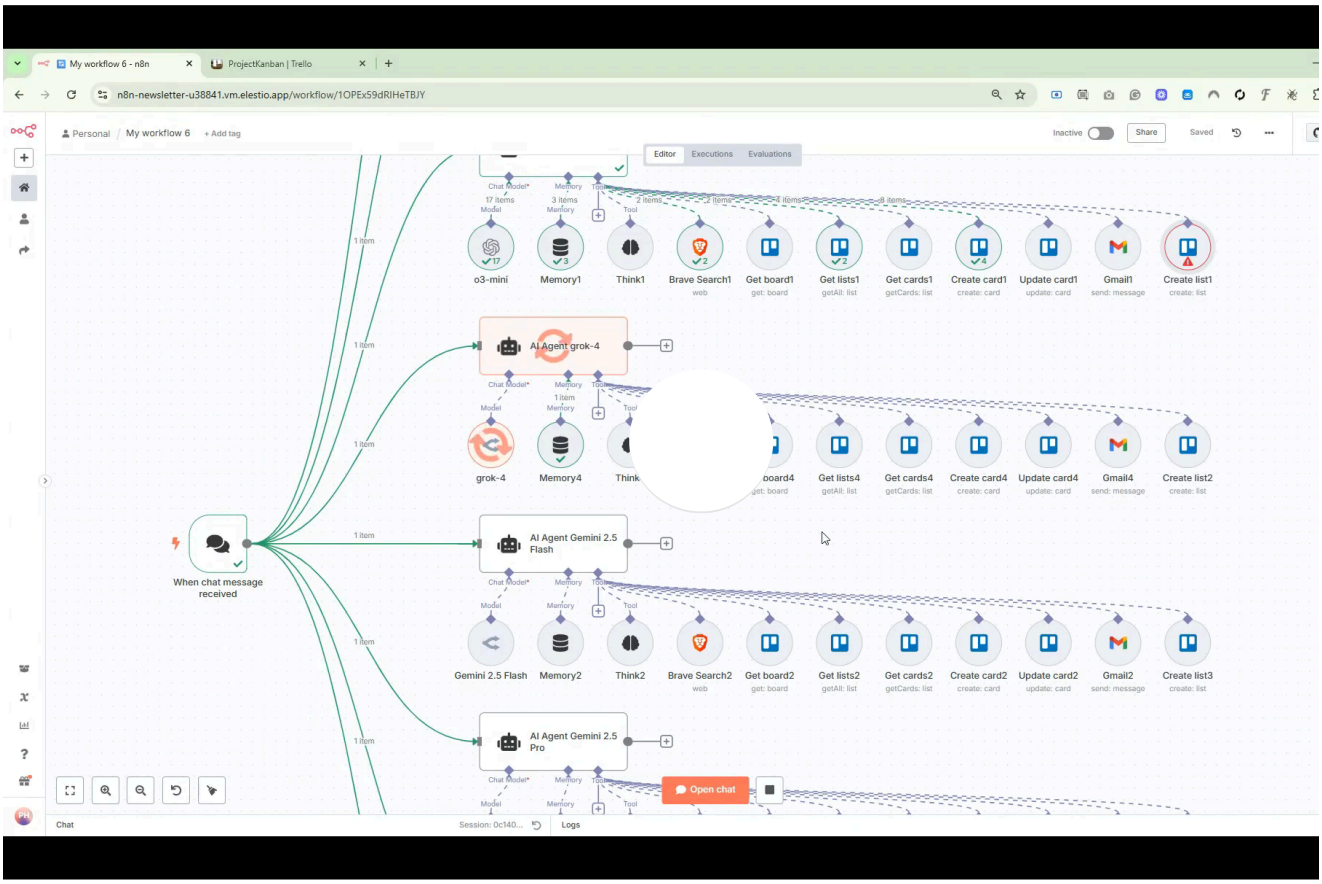
The Product Compass Newsletter

Which LLM should you use for AI Agents (updated 11/2/2025)

Of course it was just a single task.

But the results align well with what I've been observing in the recent months.

One of many tests:



Three best practices:

- Use non-reasoning LLMs for planning steps by default.
- Provide a scratchpad for AI agent to make notes.
- Or use solution like Sequential Thinking MCP to control the process.

Observations:

Updated 11/2/2025

- 5 > 4.1 > Claude Sonnet 4 > Other models
- Mini models of GPT are much cheaper but perform similarly in many cases

- Above 2-3 tool calls, every model except makes mistakes
- If the process requires more than 5 tools and dynamic planning, a single AI a will most likely fail. No matter the model. You need to clearly separate the ro and move the logic to the orchestration layer.

Bottom line:

We can create great agentic workflows and multi-agent systems. They follow we defined processes powered by LLMs. Like my multi-agent research system (Anthropic's clone).

- Possible: LLM-powered process automation
- Doesn't exist yet: Fully "agentic AI"
- Essential: Orchestration (a real moat)

Conclusion

I wanted to share the best practice I learned in the last months.

I hope this helps you avoid many mistakes.

There has never been a better time to build products without a CS degree. As PM who understand business, the market, and technology we are in the best positio leverage these opportunities. For example, to add extra revenue streams.

And whether or not you build to monetize, working with AI helps you develop bet intuition.

Thanks for Reading The Product Compass Newsletter

It’s great to explore, learn, and grow together.

Have a fantastic weekend and a great week ahead,

Paweł

68 Likes · 5 Restacks

← Previous Nex

Discussion about this post

Comments Restacks

Write a comment...